

LAPORAN

DESAIN DAN ANALISIS ALGORITMA

Dosen Pengampu: Desi Anggreani, S.Kom.,M.T.,MOS.



Disusun Oleh Kelompok 4:

AL HIZRA

105841101523

AMANDA FEBRIYANTI

105841101223

PROGRAM STUDI INFORMATIKA

FAKULTAS TEKNIK

UNIVERSITAS MUHAMMADIYAH MAKASSAR

2025

BAGIAN I

ANALISIS MASALAH

1. Identifikasi Kasus Knapsack

Kasus yang digunakan adalah 0/1 Knapsack Problem, di mana setiap item (roti) dapat dipilih maksimal satu kali.

2. Alasan penggunaan Dynamic Programming

Dynamic Programming dipilih karena:

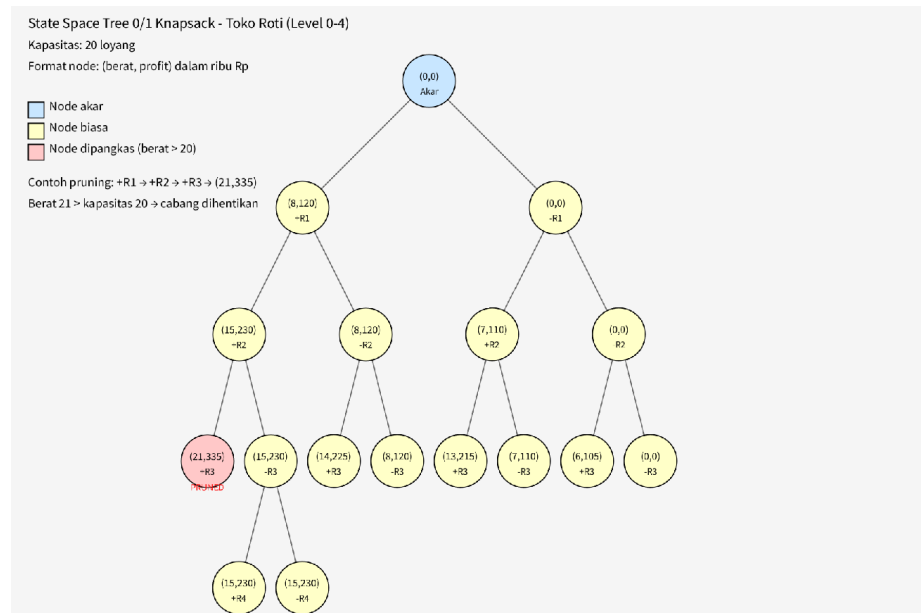
- Efisien untuk menyelesaikan masalah optimasi dengan struktur sub-masalah tumpang-tindih.
- Menghindari komputasi berulang dengan menyimpan hasil optimal dalam table DP.
- Kompleksitas waktu $O(n \times W)$, jauh lebih cepat dibanding Brute Force $O(2^n)$.

BAGIAN II

STATE SPACE TREE

1. Kerangka Simbol

- **Node**
Simpul yang mempresentasikan keadaan pemilihan item.
- **Level**
Kedalaman pohon sesuai jumlah item yang diputuskan.



- **Contoh Pruning:**
Pada level 3 dengan pilihan Roti 1, Roti 2, dan Roti 3:
Total berat = $8 + 7 + 6 = 21 > 20$.
Cabang dipangkas karena melanggar kendala kapasitas.

BAGIAN III

IMPLEMENTASI

1. Program Python

- Source Code

```
def knapsack_dp(weights, profits, capacity):  
    n = len(weights)  
    dp = [[0] * (capacity + 1) for _ in range(n + 1)]  
  
    for i in range(1, n + 1):  
        for w in range(1, capacity + 1):  
            if weights[i-1] <= w:  
                dp[i][w] = max(dp[i-1][w], dp[i-1][w  
- weights[i-1]] + profits[i-1])  
            else:  
                dp[i][w] = dp[i-1][w]  
  
    selected = []  
    w = capacity  
    for i in range(n, 0, -1):  
        if dp[i][w] != dp[i-1][w]:  
            selected.append(i-1)  
            w -= weights[i-1]  
  
    selected.reverse()  
    return dp[n][capacity], selected  
  
# Data roti  
jenis_roti = ["Roti Tawar", "Roti Gandum", "Roti  
Cokelat", "Roti Keju", "Roti Pisang",  
              "Roti Sosis", "Roti Kismis", "Roti  
Pandan", "Roti Sobek", "Roti Manis"]  
berat = [8, 7, 6, 6, 5, 7, 4, 5, 6, 4]  
keuntungan = [120000, 110000, 105000, 115000, 95000,  
130000, 85000, 90000, 100000, 80000]  
kapasitas = 20  
  
# Eksekusi
```

```

max_profit, selected_idx = knapsack_dp(berat,
keuntungan, kapasitas)

# Output
print("Kapasitas Maksimum Oven:", kapasitas,
"loyang")
print("Total Keuntungan Maksimum: Rp", max_profit)
print("Roti yang dipilih:")
total_berat = 0
for idx in selected_idx:
    print(f"    - {jenis_roti[idx]} (Berat:
{berat[idx]}, Keuntungan: Rp{keuntungan[idx]})")
    total_berat += berat[idx]
print("Total Berat Terpilih:", total_berat, "loyang")

```

- Output

```

... Kapasitas Maksimum Oven: 20 loyang
Total Keuntungan Maksimum: Rp 390000
Roti yang dipilih:
    - Roti Pisang (Berat: 5, Keuntungan: Rp95000)
    - Roti Sosis (Berat: 7, Keuntungan: Rp130000)
    - Roti Kismis (Berat: 4, Keuntungan: Rp85000)
    - Roti Manis (Berat: 4, Keuntungan: Rp80000)
Total Berat Terpilih: 20 loyang

```

BAGIAN IV

ANALISIS

1. Mengapa Dynamic Programming Optimal?

Dynamic programming memecah masalah menjadi sub-masalah yang tumpang-tindih, menyimpan hasil optimal setiap sub-masalah dalam table, sehingga menghindari perhitungan ulang. Algoritma ini menjadi solusi optimal untuk 0/1 Knapsack dengan kompleksitas $O(n \times W)$.

2. Perbandingan dengan Brute Force (konsep):

Metode	Kompleksitas Waktu
Bruto Force	$O(2^n)$
Dynamic Programming	$O(n \times W)$

Dynamic processing jauh lebih efisien karena menghindar perhitungan ulang dan langsung fokus pada solusi optimal melalui pendekatan Bottom-Up.